

BIGQUERY Practical Exercise

Q1. WHERE Clause : Filter all transactions that occurred in the year 2023.

i. SQL Query

```
1 SELECT
2   `Big-Query Practical`.`Transaction ID`,
3   `Big-Query Practical`.`Date`,
4   `Big-Query Practical`.`Customer ID`,
5   `Big-Query Practical`.`Gender`,
6   `Big-Query Practical`.`Age`,
7   `Big-Query Practical`.`Product Category`,
8   `Big-Query Practical`.`Quantity`,
9   `Big-Query Practical`.`Price per Unit`,
10  `Big-Query Practical`.`Total Amount`
11 FROM
12   `tidal-timer-493319-f9.19_Oct87.Big-Query Practical` AS `Big-Query Practical`
13 WHERE
14   EXTRACT(YEAR FROM `Big-Query Practical`.`Date`) = 2023
15 ORDER BY
16   `Big-Query Practical`.`Date` ASC;
```

ii. Results Table

Query results

Chat

Save results

Open in

Job information

Results

Visualization

JSON

Execution details

Execution graph

Row	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price per
1	522	2023-01-01	CUST522	Male	46	Beauty	3	
2	559	2023-01-01	CUST559	Female	40	Clothing	4	
3	180	2023-01-01	CUST180	Male	41	Clothing	3	
4	163	2023-01-02	CUST163	Female	64	Clothing	3	
5	303	2023-01-02	CUST303	Male	19	Electronics	3	
6	421	2023-01-02	CUST421	Female	37	Clothing	3	
7	979	2023-01-02	CUST979	Female	19	Beauty	1	
8	610	2023-01-03	CUST610	Female	26	Beauty	2	
9	231	2023-01-04	CUST231	Female	23	Clothing	3	
10	32	2023-01-04	CUST032	Male	30	Beauty	3	

Results per page: 501 – 50 of 998

Q2. Filtering + Conditions : Display all transactions where the Total Amount is more than the average Total Amount of the entire dataset.

i. SQL Query

 Q2 Filtering + Conditions    Down

```
1 SELECT
2     transactions.`Transaction ID`,
3     transactions.`Date`,
4     transactions.`Customer ID`,
5     transactions.`Gender`,
6     transactions.`Age`,
7     transactions.`Product Category`,
8     transactions.`Quantity`,
9     transactions.`Price per Unit`,
10    transactions.`Total Amount`
11 FROM
12     `tidal-timer-493319-f9.19_Oct87.Big-Query Practical` AS transactions
13 WHERE
14     transactions.`Total Amount` > (
15         SELECT
16             AVG(`Total Amount`)
17         FROM
18             `tidal-timer-493319-f9.19_Oct87.Big-Query Practical`
19     )
20 ORDER BY
21     transactions.`Total Amount` DESC;
```

ii. Results Table

Query results

Chat

Save results

Open in

Job information	Results	Visualization	JSON	Execution details	Execution graph	
Row	Transaction ID	Date	Customer ID	Gender	Age	Product Category
1	93	2023-07-14	CUST093	Female	35	Beauty
2	74	2023-11-22	CUST074	Female	18	Beauty
3	927	2023-06-24	CUST927	Male	43	Electronics
4	875	2023-08-06	CUST875	Female	51	Electronics
5	773	2023-07-23	CUST773	Male	25	Electronics
6	742	2023-01-21	CUST742	Female	38	Electronics
7	700	2023-12-09	CUST700	Male	36	Electronics
8	634	2023-10-08	CUST634	Male	60	Electronics
9	416	2023-02-17	CUST416	Male	53	Electronics
10	412	2023-09-16	CUST412	Female	19	Electronics
11	157	2023-06-24	CUST157	Male	62	Electronics

Results per page:

50

1 – 50 of 350

Q3 : Aggregate Functions: Calculate the total revenue (sum of Total Amount).

i. **SQL Query**

```
Q3 Aggregate Functions [Run] [Save query] [Download]
1 SELECT
2   SUM(transactions.`Total Amount`) AS total_revenue
3 FROM
4   `tidal-timer-493319-f9.19_Oct87.Big-Query Practical` AS transactions;
```

ii. **Results Table**

Query results		
Job information Results Visualization		
Row	total_revenue	
1	456000	

Q4. DISTINCT : Display all distinct Product Categories in the dataset.

i. **SQL Query**

```
Q4 DISTINCT [Run] [Save query] [Download] [Share]
1 SELECT DISTINCT
2   transactions.`Product Category`
3 FROM
4   `tidal-timer-493319-f9.19_Oct87.Big-Query Practical` AS transactions;
```

ii. **Results Table**

Query results		
Job information Results Visualization JSON		
Row	Product Category ▾	
1	Beauty	
2	Clothing	
3	Electronics	

Q5. GROUP BY : For each Product Category, calculate the total quantity sold.

i. SQL Queries

```

Q5 GROUP BY
Run Save query Download
1 SELECT
2     transactions.`Product Category` AS Product_Category,
3     SUM(transactions.`Quantity`) AS Total_Quantity
4 FROM
5     `tidal-timer-493319-f9.19_Oct87.Big-Query Practical` AS transactions
6 GROUP BY
7     Product_Category;

```

ii. Results Table

Query results				
Job information Results Visualization JSON Ex				
Row	Product_Category ▾	Total_Quantity ▾		
1	Beauty	771		
2	Clothing	894		
3	Electronics	849		

Q6. CASE Statement : Create a column called Age_Group that classifies customers as:

- 'Youth' (<30)
- 'Adult' (30–59)
- 'Senior' (60+)

i. SQL Query

Q6 CASE Statement

Run

Save query

Download

```

1 SELECT DISTINCT
2   transactions.`Customer ID` AS Customer_ID,
3   transactions.`Age`,
4   CASE
5     WHEN transactions.`Age` < 30 THEN 'Youth'
6     WHEN transactions.`Age` BETWEEN 30 AND 59 THEN 'Adult'
7     WHEN transactions.`Age` >= 60 THEN 'Senior'
8   END AS Age_Group
9 FROM
10  `tidal-timer-493319-f9.19_Oct87.Big-Query Practical` AS transactions
11 ORDER BY
12  transactions.`Age` ASC;

```

ii. Results Table

Q6 CASE Statement

Run

Save query

Download

Share

Schedule

This query will process 16.6 KB when run.

Query results

Chat

Save results

Options

Job information

Results

Visualization

JSON

Execution details

Execution graph

Row	Customer_ID	Age	Age_Group
1	CUST538	18	Youth
2	CUST837	18	Youth
3	CUST679	18	Youth
4	CUST022	18	Youth
5	CUST595	18	Youth
6	CUST326	18	Youth
7	CUST556	18	Youth
8	CUST058	18	Youth
9	CUST469	18	Youth
10	CUST847	18	Youth
11	CUST530	18	Youth

Results per page:

50

1 – 50 of 1000

Q7. Conditional Aggregation: For each Gender, count how many high-value transactions occurred (where Total Amount > 500).

i. SQL Query

Conditional Aggregation Run Save query Download

```
1 SELECT
2   `Big-Query Practical`.`Gender` AS Gender,
3   COUNT(*) AS High_Value_Transactions
4 FROM
5   `tidal-timer-493319-f9.19_Oct87.Big-Query Practical` AS `Big-Query Practical`
6 WHERE
7   `Big-Query Practical`.`Total Amount` > 500
8 GROUP BY Gender;
```

ii. Results Table

Query results				
Job information		Results	Visualization	JSON
Row	Gender	High_Value_Transactions		
1	Female	155		
2	Male	144		

Q8. HAVING Clause : For each Product Category, show only those categories where the total revenue exceeds 5,000.

i. SQL Query

Q8 HAVING Clause

```

1 SELECT
2     `Big-Query Practical`.`Product Category` AS Product_Category,
3     SUM(`Big-Query Practical`.`Total Amount`) AS Total_Revenue
4 FROM
5     `tidal-timer-493319-f9.19_Oct87.Big-Query Practical` AS `Big-Query Practical`
6 GROUP BY
7     Product_Category
8 HAVING
9     Total_Revenue > 5000;

```

ii. Results Table

Query results			
Job information	Results	Visualization	JSON
Row	Product_Category	Total_Revenue	
1	Beauty	143515	
2	Clothing	155580	
3	Electronics	156905	

Q9. Calculated Fields: Display a new column called Unit_Cost_Category that labels a transaction as:

- 'Cheap' if Price per Unit < 50
- 'Moderate' if Price per Unit between 50 and 200
- 'Expensive' if Price per Unit > 200

i. SQL Query

Q9 Calculated Fields

```

1 SELECT
2     t.`Transaction ID`,
3     t.`Price per Unit`,
4     CASE
5         WHEN t.`Price per Unit` < 50 THEN 'Cheap'
6         WHEN t.`Price per Unit` BETWEEN 50 AND 200 THEN 'Moderate'
7         WHEN t.`Price per Unit` > 200 THEN 'Expensive'
8     END AS Unit_Cost_Category
9 FROM
10    `tidal-timer-493319-f9.19_Oct87.Big-Query Practical` AS t;

```

ii. Results Table

Query results					Chat	Save results ▼
Job information		Results	Visualization	JSON	Execution details	Execution graph
Row	Transaction ID	Price per Unit	Unit_Cost_Category			
1	191	25	Cheap			
2	204	25	Cheap			
3	230	25	Cheap			
4	232	25	Cheap			
5	309	25	Cheap			
6	310	25	Cheap			
7	363	25	Cheap			
8	371	25	Cheap			
9	397	25	Cheap			
10	454	25	Cheap			
11	512	25	Cheap			
12	700	25	Cheap			

Results per page: 50 ▼ 1 – 50 of 1000

Q10. Combining WHERE + CASE : Display all transactions from customers aged 40 or older and add a column Spending_Level showing:

- 'High' if Total Amount > 1000
- 'Low' otherwise

i. SQL Query

```
Q10 Combining WHERE + CASE Run Save query

1 SELECT
2     t.`Customer ID`,
3     t.`Age`,
4     t.`Total Amount`,
5     CASE
6         WHEN t.`Total Amount` > 1000 THEN 'High'
7         ELSE 'Low'
8     END AS Spending_Level
9 FROM
10     `tidal-timer-493319-f9.19_Oct87.Big-Query Practical` AS t
11 WHERE
12     t.`Age` >= 40;
```

ii. Results Table

Query results Chat Save results

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	Customer ID	Age	Total Amount	Spending_Level	
1	CUST191	64	25	Low	
2	CUST230	54	25	Low	
3	CUST232	43	25	Low	
4	CUST363	64	25	Low	
5	CUST454	46	25	Low	
6	CUST512	57	25	Low	
7	CUST791	51	25	Low	
8	CUST825	46	25	Low	
9	CUST855	54	25	Low	
10	CUST967	62	25	Low	
11	CUST006	45	30	Low	

Results per page: 50 1 - 50 of 558

Save results ▼



Results

JSON

Execution graph

Row	Customer ID	Age	Total Amount	Spending_Level	
1	CUST191	64	25	Low	
2	CUST230	54	25	Low	
3	CUST232	43	25	Low	
4	CUST363	64	25	Low	
5	CUST454	46	25	Low	
6	CUST512	57	25	Low	
7	CUST791	51	25	Low	
8	CUST825	46	25	Low	
9	CUST855	54	25	Low	
10	CUST967	62	25	Low	
11	CUST006	45	30	Low	

Results per page: 50 ▼ 1 – 50 of 558 |◀